PROJECT TECHNICAL REPORT

ON

SLIDING WINDOW PROTOCOL IN COMPUTER NETWORKS

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

by

Kanamukkala Suhail(24261A6625)

Under the guidance of

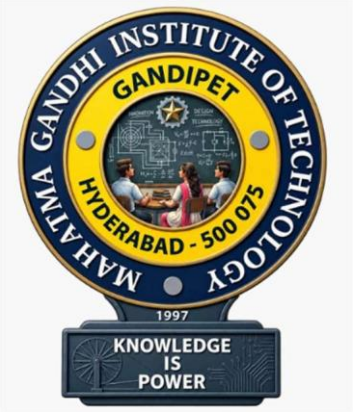
Dr. C. S. Srinivas

Department of M&H

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad, A.P.)

Chaitanya Bharathi P.O., Gandipet, Hyderabad - 500075



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)

Hyderabad, Telangana

2026

Declaration

I declare that the project report "**SLIDING WINDOW PROTOCOL IN COMPUTER NETWORKS**," submitted to the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Mahatma Gandhi Institute of Technology (A), Hyderabad, for the award of the Degree of **Bachelor of Technology**, is an original work carried out by me.

I also declare that all the sources referred to in this report have been acknowledged and that any omissions or errors are entirely inadvertent.

Kanamukkala Suhail

Signed this day, 09thNovemeber,2026

CERTIFICATE

Dr. C. S. Srinivas
Department of M&H
Mahatma Gandhi Institute of Technology
Gandipet, Telangana 500075

Certified that the technical report, "**SLIDING WINDOW PROTOCOL IN COMPUTER NETWORKS**" submitted to the Mahatma Gandhi Institute of Technology for the award of **Bachelor of Technology degree in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** is based on independent and bonafide research work carried out by **Kanamukkala Suhail**

It is further certified that this project work has been completed successfully and has not been submitted to any other university or institution for the award of any degree or diploma.

Signed this day, 09th November, 2026

Dr. C. S. Srinivas
(Assistant Professor)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Dr. C. S. Srinivas**, for continuous encouragement, valuable technical guidance, and constructive suggestions throughout the preparation of this report. The discussions on communication channels, coding methods, and protocol design helped me organise the subject and understand the practical purpose of error control in computer networks.

I also thank the Head of the Department and the faculty members of the Department of Computer Science and Engineering (AI & ML), Mahatma Gandhi Institute of Technology (A), for providing academic guidance, access to learning resources, and a supportive environment for completing this work. Their feedback helped improve the clarity and technical organisation of the report.

My special thanks go to my friends and classmates for their cooperation, discussions, and suggestions while reviewing the worked examples and software-oriented design. Their support helped me identify areas that required clearer explanation, especially the distinction between error detection, correction, and retransmission-based recovery.

Finally, I express my heartfelt gratitude to my family for their patience, motivation, and encouragement throughout the preparation of this report. Their constant support played an important role in completing this work.

KANAMUKKALA SUHAIL

(24261A6625)

ABSTRACT

Reliable data transfer is an important requirement in computer networks. A sender must control the amount of data in transit, preserve frame order, and recover from lost or delayed frames. Stop-and-Wait transmission permits only one outstanding frame and can underutilise a link when the round-trip delay is large. The Sliding Window Protocol improves utilisation by allowing multiple frames to be transmitted before acknowledgements are received.

This report studies the basic sliding-window mechanism and its major ARQ variants: Go-Back-N and Selective Repeat. It explains sender and receiver windows, sequence numbers, cumulative and individual acknowledgements, timers, retransmission, and sequence-number wraparound. A software-oriented simulation design is presented to demonstrate normal transmission and frame-loss recovery. The report compares the protocols in terms of receiver complexity, buffering, retransmission overhead, and suitability for different communication conditions. It also relates the frame-oriented concepts to TCP, which uses byte sequence numbers, cumulative acknowledgements, and an advertised receive window.

Keywords: Sliding Window Protocol, Stop-and-Wait, Go-Back-N, Selective Repeat, ARQ, sequence number, acknowledgement, flow control, TCP.

TABLE OF CONTENTS

CONTENTS	PAGE
Declaration	**2**
Certificate	**3**
Acknowledgement	**4**
Abstract	**5**
Table of Contents	**6**
List of Figures and Tables	**8**
Chapter 1: Introduction	**9-10**
1.1 Background	9
1.2 Motivation	9
1.3 Problem Statement	9
1.4 Objectives	9
1.5 Scope of the Project	10
1.6 Limitations	10
Chapter 2: Literature Review	**11**
2.1 Development of Reliable Data Transfer	11
2.2 Go-Back-N ARQ	11
2.3 Selective Repeat ARQ	11
2.4 Sliding Windows in TCP	11
2.5 Review Summary	12
Chapter 3: Methodology and System Design	**13**
3.1 System Overview	13
3.2 Sliding Window Fundamentals	13
3.3 Stop-and-Wait ARQ	14
3.4 Go-Back-N ARQ	15
3.5 Selective Repeat ARQ	15
3.6 Proposed Workflow	16
3.7Protocol Case Study: TCP	16
Chapter 4: Implementation and Experimental Work	**18**

4.1 Software Demonstration Design	18
4.2 Algorithm: Go-Back-N	18
4.3 Algorithm: Selective Repeat	18
4.4 Sample Python Simulation	18
4.5 Test Cases	19
4.6 Experimental Parameters	19
Chapter 5: Results and Discussion	**20**
5.1 Functional Outcomes	20
5.2 Comparative Analysis	20
5.3 Illustrative Example	20
5.4 Discussion	20
Chapter 6: Advantages and Applications	**21**
6.1 Advantages	21
6.2 Applications	21
Chapter 7: Conclusion and Future Scope	**22**
7.1 Conclusion	22
7.2 Future Scope	22
Chapter 8: References	**23**
Chapter 9: Appendix	**24**
Chapter 10: Bibliography	**26**

LIST OF FIGURES AND TABLES

LIST OF FIGURES

NO.	TITLE	PAGE
Fig. 1	Sender window before and after acknowledgement	14
Fig. 2	Go-Back-N frame-loss and retransmission sequence	15
Fig. 3	Selective Repeat buffering and selective retransmission	15
Fig. 4	Sliding-window simulation workflow	16
Fig. 5	Basic sender–channel–receiver model	17

LIST OF TABLES

NO.	TITLE	PAGE
Table 1	Sequence-number window constraints	14
Table 2	Comparative Analysis of ARQ Protocols	17
Table 3	Sample Simulation Test Cases	19

CHAPTER 1: INTRODUCTION

1.1 Background

Computer networks transfer information by dividing a message into smaller units such as frames, packets, or segments. A sender must regulate how much data it transmits before receiving confirmation from the receiver. If it waits after every frame, the communication channel may remain idle for much of the time, especially when propagation delay is high. If it sends continuously without control, the receiver may be overwhelmed and lost data may be difficult to recover.

The Sliding Window Protocol is a flow-control and error-control mechanism that allows a sender to transmit multiple frames before receiving acknowledgements. The sender maintains a window representing the sequence numbers that may be sent or are awaiting acknowledgement. As acknowledgements arrive, the window advances, or "slides", to make room for new frames. The receiver also maintains a window that describes which frames it is prepared to accept.

Sliding-window techniques are used in reliable data-link protocols and are reflected in the operation of transport protocols. TCP uses byte sequence numbers, acknowledgements, and an advertised receive window. Its acknowledgement number is cumulative: it identifies the next byte expected by the receiver [1].

1.2 Motivation

Stop-and-Wait allows only one outstanding frame. It is simple, but a sender may spend considerable time waiting for an acknowledgement after every transmission. Sliding windows improve link utilisation by keeping several frames in transit. This is particularly useful on long-distance, high-bandwidth, or otherwise high-delay links.

Two widely studied ARQ variants are Go-Back-N and Selective Repeat. Go-Back-N uses cumulative acknowledgements and retransmits the missing frame together with later outstanding frames. Selective Repeat buffers correctly received out-of-order frames and retransmits only the frames that are missing or damaged. The choice involves a trade-off between implementation simplicity, buffering, retransmission overhead, and throughput.

1.3 Problem Statement

A reliable communication system must transfer data in order, prevent the sender from overrunning the receiver, identify missing or damaged frames, and recover from transmission failures. A Stop-and-Wait approach can underuse the available channel when round-trip delay is significant. The project studies how a sliding window improves utilisation while maintaining reliable delivery through sequence numbers, acknowledgements, timers, and retransmissions.

1.4 Objectives

- Explain the principle and operation of the Sliding Window Protocol.
- Study sender and receiver windows, sequence numbers, acknowledgements, and timers.
- Compare Stop-and-Wait, Go-Back-N, and Selective Repeat ARQ.
- Demonstrate frame transmission, acknowledgement processing, loss, and retransmission using a software simulation.

• Evaluate the protocols using successful delivery, retransmission count, and illustrative link utilisation.

• Relate sliding-window concepts to TCP flow control and reliable data transfer.

1.5 Scope of the Project

The project focuses on a teaching-level simulation of a reliable point-to-point link. It models numbered frames, a finite sequence-number space, a sender window, a receiver window, acknowledgements, frame loss, and retransmission timers. Go-Back-N and Selective Repeat are considered in detail, with Stop-and-Wait used as a baseline. The report also explains how TCP uses a byte-oriented sliding receive window. The simulation is not intended to be a complete implementation of TCP or a standards-compliant network stack.

1.6 Limitations

The simulation uses simplified timing and deterministic or manually selected loss events. It does not model physical-layer noise, congestion-control algorithms, packet checksums, routing, operating-system socket buffers, or real network measurements. Results depend on the selected window size, timeout, propagation delay, and loss pattern. TCP is more complex than the basic ARQ protocols presented here: it uses byte sequence numbers, cumulative acknowledgements, retransmission timers, congestion control, and additional recovery mechanisms [1].

CHAPTER 2: LITERATURE REVIEW

2.1 Development of Reliable Data Transfer

Reliable data transfer protocols were developed to cope with unreliable communication channels in which frames may be corrupted, lost, duplicated, or delivered out of order. Early elementary protocols use acknowledgements and retransmission to ensure that data is not silently lost. Stop-and-Wait is the simplest approach: one frame is sent and the sender waits for its acknowledgement before sending the next frame. Sequence numbers are required to distinguish a new frame from a retransmitted duplicate.

The sliding-window family generalises this approach by permitting multiple unacknowledged frames. The sender tracks the oldest unacknowledged frame and the next frame eligible for transmission. Acknowledgements move the lower edge of the sender window forward. This improves utilisation because transmission can continue while earlier frames are travelling through the network.

2.2 Go-Back-N ARQ

Go-Back-N is a pipelined automatic repeat request protocol. The sender may transmit up to N frames without waiting for individual acknowledgements. The receiver generally accepts only the next expected frame and discards out-of-order frames. Acknowledgements are cumulative: an acknowledgement for frame k indicates that all frames before k have been received in order, with the precise numbering convention defined by the protocol. If a frame is lost or its timer expires, the sender retransmits that frame and all later outstanding frames.

For an m-bit sequence-number field, the sequence-number space contains 2^m values. In the conventional Go-Back-N design, the maximum sender window is 2^m − 1. This restriction prevents ambiguity between old outstanding frames and new frames after sequence numbers wrap around.

2.3 Selective Repeat ARQ

Selective Repeat improves efficiency by allowing the receiver to accept and buffer frames that arrive out of order, provided they fall within its receive window. Each frame can be acknowledged separately. When a frame is lost, the sender retransmits only that frame rather than resending every later frame. The receiver stores out-of-order frames and delivers them to the upper layer when the missing earlier frames arrive.

Selective Repeat requires more receiver memory and more complex acknowledgement and timer management. To avoid ambiguity when sequence numbers wrap around, the sender and receiver windows are normally limited to at most half the sequence-number space. With m sequence bits, the maximum window size is 2^(m−1).

2.4 Sliding Windows in TCP

TCP provides a byte-oriented sliding-window mechanism. Each data byte occupies sequence space, and acknowledgements are cumulative. The receiver advertises a receive window to communicate how many additional bytes it is prepared to accept. The sender uses the advertised window to limit outstanding data and avoid overflowing the receiver's buffer. TCP sequence numbers are 32 bits and are handled modulo 2^32 [1]. TCP's window mechanism is related to, but not identical with, the textbook frame-oriented Go-Back-N and Selective Repeat examples.

2.5 Review Summary

The reviewed concepts show that sliding windows solve two related problems: controlling the amount of outstanding data and supporting recovery from loss. Go-Back-N is simpler but may retransmit frames that were already received. Selective Repeat can reduce unnecessary retransmissions but needs buffering and more state. TCP applies a sliding receive window to a byte stream and combines it with cumulative acknowledgements and other reliability mechanisms. These differences form the basis for the comparative study in this report.

CHAPTER 3: METHODOLOGY AND SYSTEM DESIGN

3.1 System Overview

The proposed model consists of a sender, an unreliable communication channel, and a receiver. The sender divides input data into frames and assigns sequence numbers. It stores transmitted but unacknowledged frames in a retransmission buffer. The channel may deliver a frame, lose it, or delay it. The receiver checks the sequence number, accepts frames according to its receive-window rule, and sends acknowledgements. On receiving acknowledgements, the sender advances its window. If an acknowledgement does not arrive before the timer expires, the sender retransmits according to the selected ARQ method.

3.1.1 Main Components

Sender: maintains the send base, next sequence number, window size, outstanding-frame buffer, and timer information.

Communication channel: represents transmission delay and possible frame or acknowledgement loss.

Receiver: maintains the next expected sequence number or a receive window, stores accepted data, and generates acknowledgements.

Timer: detects that an outstanding frame has not been acknowledged within the configured interval.

Sequence-number field: labels frames so that the receiver can identify order and duplicates.

3.2 Sliding Window Fundamentals

Let the sequence-number space contain S = 2^m values, where m is the number of sequence bits. Sequence numbers are used cyclically from 0 to S−1. The sender window defines the range of frames that may be transmitted without waiting for further acknowledgements. The receiver window defines the sequence numbers it is currently prepared to accept.

For a sender window of size W, up to W frames may be outstanding at one time. When an acknowledgement confirms receipt of one or more frames, the sender moves the window forward. The newly available positions can then be used for new frames. Window movement is circular when the sequence-number space wraps around.

In Go-Back-N, the receiver window is normally one frame. In Selective Repeat, the receiver window may contain multiple sequence numbers and out-of-order frames can be buffered.

3.2.1 Sequence Number Constraints

For Go-Back-N with m sequence bits, the sender window must satisfy $W \leq 2^m - 1$. For Selective Repeat, both sender and receiver windows are conventionally limited to $W \leq 2^{(m-1)}$. These limits prevent a delayed duplicate from being mistaken for a new frame after sequence-number reuse. For example, with m = 3, the sequence numbers are 0 through 7. Go-Back-N can use a sender window of at most 7, while Selective Repeat can use windows of at most 4.

Protocol	Window Constraint	Reason
Go-Back-N	$W \leq 2^m - 1$	Avoid sequence-number ambiguity when ACKs are delayed or lost.
Selective Repeat	$W \leq 2^{(m-1)}$	Separate old and new window bounds, avoiding overlap during wraparound.

Table 1: Sequence-number window constraints

3.2.2 Worked Window Example

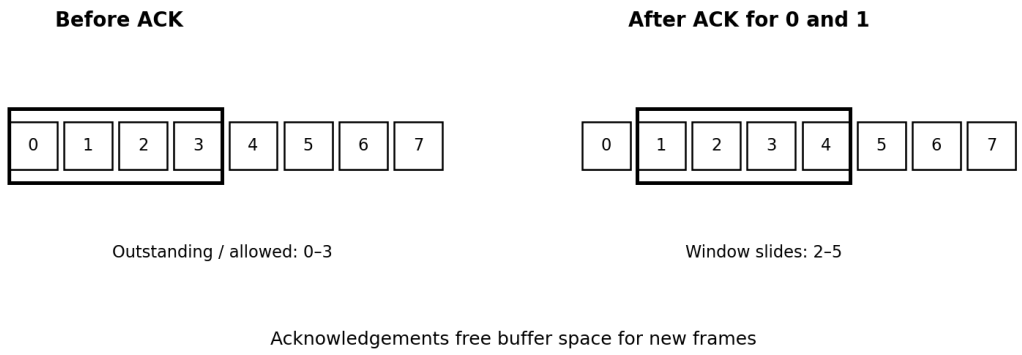


Fig. 1 Sender window before and after acknowledgement

Assume a 3-bit sequence-number field and a sender window size of 4. The sequence numbers are 0, 1, 2, 3, 4, 5, 6, and 7. Initially, the sender may transmit frames 0, 1, 2, and 3. If acknowledgements confirm frames 0 and 1, the window advances and the sender may transmit frames 4 and 5. After frame 7, numbering wraps around to 0. Correct window limits and acknowledgement processing are essential to distinguish new frames from old duplicates.

3.3 Stop-and-Wait ARQ

Stop-and-Wait sends one frame and waits for its acknowledgement. If the acknowledgement arrives, the sender transmits the next frame. If the frame or acknowledgement is lost, a timeout causes retransmission. A one-bit sequence number (0 or 1) is sufficient to distinguish a new frame from a retransmitted copy. The method is easy to implement but can be inefficient when the round-trip time is large compared with the frame transmission time.

3.4 Go-Back-N ARQ

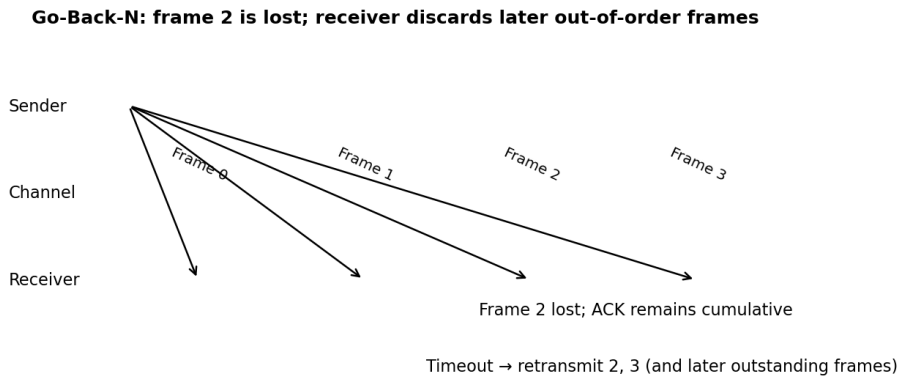


Fig. 2 Go-Back-N frame-loss and retransmission sequence

Go-Back-N permits several frames to be in transit. The sender uses a single timer for the oldest unacknowledged frame in the common textbook model. The receiver accepts only the next expected frame and sends a cumulative acknowledgement. If frame 2 is lost while frames 3 and 4 arrive, the receiver discards frames 3 and 4 because frame 2 is still missing. When the timer expires, the sender retransmits frame 2 and all later outstanding frames.

Advantages include simple receiver logic and cumulative acknowledgements. The main disadvantage is redundant retransmission when later frames have already crossed the channel successfully.

3.5 Selective Repeat ARQ

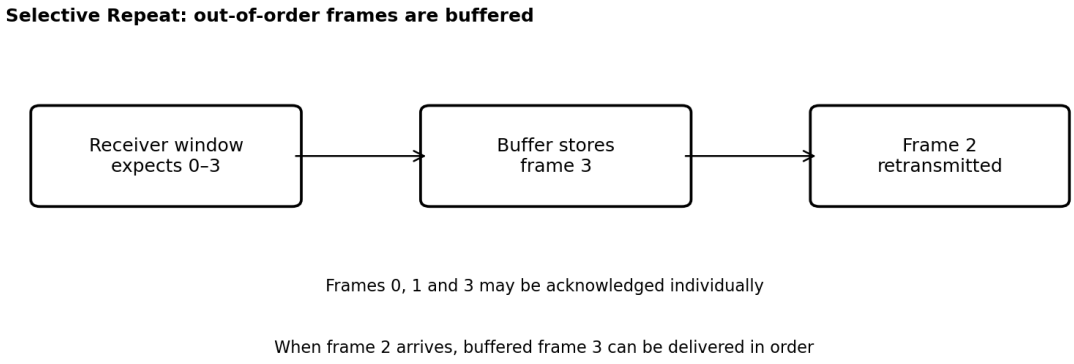


Fig. 3 Selective Repeat buffering and selective retransmission

Selective Repeat permits the receiver to accept frames that arrive out of order but lie within its receive window. It buffers these frames and acknowledges them individually. The sender maintains per-frame acknowledgement status and typically a timer for each outstanding frame. If frame 2 is lost but frames 3 and 4 arrive, the receiver can buffer 3 and 4 and acknowledge them. The sender retransmits frame 2 only. Once frame 2 arrives, the receiver can deliver frames 2, 3, and 4 in order.

Selective Repeat can reduce retransmission overhead on lossy links. Its costs are greater buffer requirements, more complex state management, and careful sequence-number window constraints.

3.6 Proposed Workflow

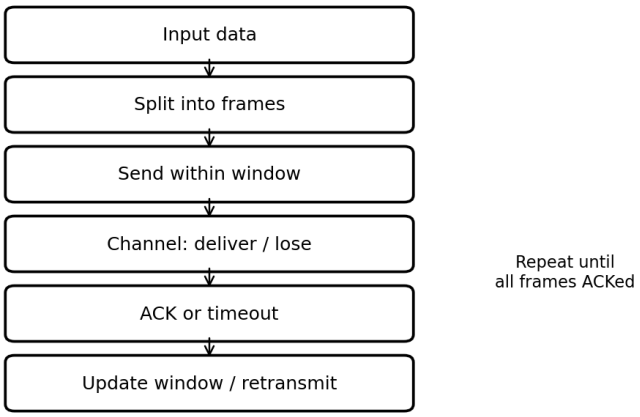


Fig. 4 Sliding-window simulation workflow

1. Read the message and divide it into fixed-size frames.
2. Initialise sequence numbers, sender base, receiver state, and window size.
3. Transmit all eligible frames inside the sender window.
4. Simulate channel delivery, loss, or delay.
5. At the receiver, check whether each arriving frame is acceptable.
6. Generate cumulative or individual acknowledgements according to the protocol.
7. Update the sender window when valid acknowledgements arrive.
8. If a timer expires, retransmit according to Go-Back-N or Selective Repeat rules.
9. Continue until all frames are delivered and acknowledged, or a retry limit is reached.

3.7 Protocol Case Study: TCP

TCP uses a sliding receive window to manage the amount of data a receiver is willing to accept. Its sequence numbers identify bytes rather than individual application messages. An acknowledgement number indicates the next byte expected, thereby acknowledging all earlier contiguous bytes. The advertised window communicates available receive capacity. TCP also uses retransmission and congestion-control mechanisms; the advertised receive window is flow control, while the congestion window is a separate sender-side limit [1].

Feature	**Stop-and-Wait**	**Go-Back-N**	**Selective Repeat**
Outstanding Frames	1	Up to W ($W \leq 2^m - 1$)	Up to W ($W \leq 2^{m-1}$)
Receiver Buffering	Minimal (1	None (Discards out-of-	Required (Buffers out-of-

Feature	Stop-and-Wait	Go-Back-N	Selective Repeat
	frame)	order)	order)
ACK Style	Per frame	Cumulative	Individual / Selective
Retransmission	Current frame only	Missing frame onward	Only missing frame
Implementation Complexity	Low	Moderate	High

Table 2: Comparative Analysis of ARQ Protocols

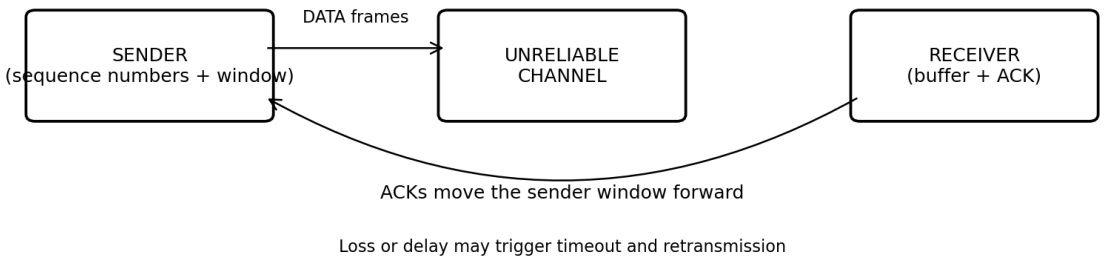


Fig. 5 Basic sender–channel–receiver model

CHAPTER 4: IMPLEMENTATION AND EXPERIMENTAL WORK

4.1 Software Demonstration Design

A Python simulation can demonstrate the movement of sender and receiver windows and compare retransmission behaviour. The program accepts a list of frames, a window size, a protocol selection, and a set of frame-loss events. It records transmissions, acknowledgements, accepted frames, discarded frames, and retransmissions. The model uses a simplified event sequence rather than real sockets, so it is suitable for illustrating protocol logic in a controlled environment.

4.2 Algorithm: Go-Back-N

Initialise base = 0 and next_seq = 0.

While unacknowledged frames remain:

While next_seq is inside the sender window and unsent frames remain:

transmit frame next_seq; start timer if this is the oldest outstanding frame; increment next_seq.

Process arriving frames at the receiver:

if frame equals the next expected sequence, accept it and advance expected;

otherwise discard it and repeat the cumulative ACK for the next expected frame.

On cumulative ACK, slide base forward over all acknowledged frames.

On timeout, retransmit every outstanding frame from base through next_seq − 1.

Stop when every frame has been acknowledged.

4.3 Algorithm: Selective Repeat

Initialise sender and receiver windows and an acknowledgement flag for every frame.

Transmit each unsent frame inside the sender window and start its timer.

When a frame arrives:

if it is inside the receiver window and not already received, buffer it and send its individual ACK;

if it is a duplicate, send the ACK again;

otherwise discard it.

Deliver buffered frames in order whenever the next expected frame is available.

When an individual ACK arrives, mark that frame acknowledged.

Slide the sender window across consecutively acknowledged frames.

On a frame timer expiry, retransmit only that unacknowledged frame.

Stop when every frame has been acknowledged.

4.4 Sample Python Simulation

The following compact example demonstrates the central distinction between the two ARQ strategies. It models one loss event and counts data-frame transmissions. It is an educational event model; a full implementation should also model acknowledgement loss, propagation time, timer scheduling, and sequence-number wraparound.

4.5 Test Cases

Test ID	Scenario Description	Expected Go-Back-N Behavior	Expected Selective Repeat Behavior
Test A	No frame loss	Sequential transmission; 0 retransmissions.	Sequential transmission; 0 retransmissions.
Test B	Single frame loss (Frame 1)	Discards Frame 2 & 3; retransmits 1, 2, 3.	Buffers Frame 2 & 3; retransmits Frame 1 only.
Test C	ACK loss	Cumulative ACK recovers pipeline without extra retransmissions.	Timeout triggers retransmission of missing ACK frame.
Test D	Sequence wraparound	Wraps 7 → 0 correctly within W ≤ 7.	Wraps 7 → 0 correctly within W ≤ 4.

Table 3: Sample Simulation Test Cases

4.6 Experimental Parameters

The main parameters are number of frames, sender window size, sequence-number bits, frame transmission time, round-trip propagation time, timeout value, and the locations of lost frames. A fair comparison keeps the frame set and loss pattern identical for both protocols. Useful outputs include total data-frame transmissions, retransmissions, successfully delivered frames, completion time, and channel utilisation. The report does not claim measured performance from a live network.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Functional Outcomes

The expected functional outcome is that both Go-Back-N and Selective Repeat eventually deliver all frames when the channel becomes available and retransmission is permitted. With no loss, both protocols can keep multiple frames outstanding up to the configured sender window. When an early frame is lost, Go-Back-N retransmits that frame and later outstanding frames. Selective Repeat buffers later frames and retransmits only the missing frame.

These outcomes illustrate the principal trade-off: Go-Back-N has simpler receiver behaviour but can generate additional transmissions, whereas Selective Repeat reduces unnecessary retransmissions at the cost of buffering and more complex acknowledgement tracking.

5.2 Comparative Analysis

Stop-and-Wait: one outstanding frame; simplest implementation; potentially low utilisation on high-delay links.

Go-Back-N: multiple outstanding frames; cumulative acknowledgements; receiver generally accepts only the next expected frame; retransmits from the missing frame onward.

Selective Repeat: multiple outstanding frames; individual acknowledgements; receiver buffers out-of-order frames; retransmits only missing or damaged frames.

TCP sliding window: byte-oriented sequence space; cumulative acknowledgements; advertised receive window; combined with retransmission and congestion control.

5.3 Illustrative Example

Assume the sender transmits frames 0, 1, 2, 3, and frame 1 is lost. Frames 2 and 3 reach the receiver.

In Go-Back-N, the receiver is waiting for frame 1, so it does not accept frames 2 and 3 as in-order data. It repeats the acknowledgement indicating that frame 1 is expected. When the sender detects the loss, it retransmits frames 1, 2, and 3.

In Selective Repeat, the receiver accepts and buffers frames 2 and 3 if they are inside its receive window, and acknowledges them individually. The sender retransmits frame 1 only. When frame 1 arrives, the receiver can deliver the buffered frames in order. This example explains why Selective Repeat may use fewer retransmissions when losses occur.

5.4 Discussion

Window size affects performance. A very small window may leave the link idle while acknowledgements travel back. A larger window can keep more data in flight, but it requires sufficient receiver buffering and must respect sequence-number constraints. A timeout that is too short can cause unnecessary retransmissions; a timeout that is too long delays recovery after a genuine loss.

The simulation should be interpreted as a protocol demonstration, not as a benchmark of commercial network performance. Actual throughput also depends on bandwidth, propagation delay, congestion, packet size, implementation, and competing traffic. TCP's receive window and congestion window must not be confused: one protects receiver capacity, while the other controls the sender's contribution to network load [1].

CHAPTER 6: ADVANTAGES AND APPLICATIONS

6.1 Advantages

• Improves channel utilisation by allowing several frames to be transmitted before acknowledgements return.

• Provides flow control by limiting the amount of outstanding data.

• Supports reliable delivery through acknowledgements, timers, and retransmission.

• Uses sequence numbers to detect duplicates and preserve ordering.

• Allows protocol designers to trade receiver complexity against retransmission overhead.

• Forms an important conceptual basis for TCP's byte-oriented receive-window mechanism.

6.2 Applications

Sliding-window concepts are used in reliable data communication and transport protocols. TCP uses sequence numbers, cumulative acknowledgements, and a receiver-advertised window to support reliable byte-stream delivery and flow control [1]. Data-link protocols may use window-based ARQ to recover from lost or damaged frames on a link. High-bandwidth and long-delay connections benefit from pipelining because multiple data units can be in transit simultaneously. The exact protocol behaviour and window rules depend on the relevant standard or implementation.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This report studied the Sliding Window Protocol as a method for improving data-transfer efficiency while maintaining reliable communication. The sender window limits the number of outstanding frames, and acknowledgements allow the window to advance. Sequence numbers identify frames and support duplicate detection and ordering.

Stop-and-Wait is simple but may underutilise a link when round-trip delay is high. Go-Back-N improves pipelining and uses cumulative acknowledgements, but may retransmit correctly received later frames. Selective Repeat buffers out-of-order frames and retransmits only missing frames, but requires more memory and more complex state management. TCP applies related sliding-window ideas to bytes and advertises receiver capacity. The most suitable approach depends on delay, loss, buffer availability, and implementation complexity.

7.2 Future Scope

- Implement a complete interactive simulator with sender and receiver window visualisation.
- Add configurable frame loss, acknowledgement loss, corruption, and variable delay.
- Compare throughput and retransmission count over multiple loss rates and window sizes.
- Add sequence-number wraparound and duplicate-frame test scenarios.
- Extend the model to include delayed acknowledgements and adaptive retransmission timers.
- Use a network simulator such as ns-3 to study performance under realistic traffic and congestion.
- Compare the simplified ARQ models with TCP traces captured from a controlled experiment.

CHAPTER 8: REFERENCES

References

[1] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, Internet Engineering Task Force, August 2022. https://www.rfc-editor.org/rfc/rfc9293.html

[2] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed., Pearson, 2011.

[3] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021.

[4] B. A. Forouzan, Data Communications and Networking, 5th ed., McGraw-Hill, 2012.

[5] J. Touch, “Sequence Number Extension for Windowed Protocols,” RFC 9187, Internet Engineering Task Force, January 2022. https://www.rfc-editor.org/rfc/rfc9187.html

CHAPTER 9: APPENDIX

A.1 Window Movement Example

Let m = 3, so the sequence-number space is 0–7. Let the sender window size be 4. The initial sender window contains 0, 1, 2, and 3. After frames 0 and 1 are acknowledged, the window advances to 2, 3, 4, and 5. After further acknowledgements, it continues to move and eventually wraps from 7 to 0. In Selective Repeat, the receiver window must be no larger than 4 for this 3-bit sequence space.

A.2 Sample Python Code

```
from collections import deque

def go_back_n(frames, window_size, lost_once):
    base = 0
    next_to_send = 0
    expected = 0
    transmissions = []
    delivered = []
    lost = set(lost_once)

    while base < len(frames):
        # Fill the sender window
        while next_to_send < len(frames) and next_to_send < base + window_size:
            transmissions.append(next_to_send)
            next_to_send += 1

        # Receiver processes this batch in order
        batch_start = base
        batch_end = next_to_send
        for seq in range(batch_start, batch_end):
            if seq in lost:
                lost.remove(seq)       # lose it only on its first attempt
                break
            if seq == expected:
                delivered.append(frames[seq])
                expected += 1

        # Cumulative ACK advances base to the next expected frame
        new_base = expected
        if new_base == base:
            # Timeout: retransmit from the first unacknowledged frame
            for seq in range(base, next_to_send):
                transmissions.append(seq)
                if seq == expected:
                    delivered.append(frames[seq])
                    expected += 1
            new_base = expected

        base = new_base

    return delivered, transmissions

if __name__ == "__main__":
    data = ["A", "B", "C", "D", "E"]
    received, sent = go_back_n(data, window_size=3, lost_once={1})
    print("Delivered:", received)
    print("Data-frame transmissions:", sent)
    print("Total transmissions:", len(sent))
```

A.3 Suggested Output and Interpretation

For a test with no loss, each frame should be transmitted once and delivered in order. When an early frame is lost, Go-Back-N should require retransmission of that frame and any later outstanding frames that were not

accepted in order. A Selective Repeat implementation should buffer later frames and retransmit only the missing frame. Exact counts depend on the event schedule and the simulator's timeout model.

A.4 Important Formulae

Sequence-number space: $S = 2^m$

Go-Back-N sender window: $W \leq 2^m - 1$

Selective Repeat sender/receiver window: $W \leq 2^{(m-1)}$

Maximum outstanding frames: at most W

Approximate pipelining principle: a larger window can keep more frames in flight, subject to link capacity, delay, and receiver buffer limits.

CHAPTER 10: BIBLIOGRAPHY

Bibliography

[1] RFC Editor, “RFC 9293: Transmission Control Protocol (TCP),” https://www.rfc-editor.org/rfc/rfc9293.html

[2] RFC Editor, “RFC 9187: Sequence Number Extension for Windowed Protocols,” https://www.rfc-editor.org/rfc/rfc9187.html

[3] Tanenbaum, A. S., and Wetherall, D. J., Computer Networks, 5th edition, Pearson.

[4] Kurose, J. F., and Ross, K. W., Computer Networking: A Top-Down Approach, 8th edition, Pearson.

[5] Forouzan, B. A., Data Communications and Networking, 5th edition, McGraw-Hill.